

π_0 · arXiv 2410.24164 · RSS 2025

Stakeholder

18+3 · seat 2

Reference 2025–26 VLA template
OpenPI start · stack snapshot · risks

Sep 28 paper club · VLA Architectures

Spine: PaliGemma + OT-CFM action chunks (DDPM→flow)

Physical Intelligence · H=50 · ~10 Euler · ~73 ms RTX4090 · NOT a classical planner

Why π_0 is the reference VLA template

Architecture

PaliGemma + FM
action expert + chunks

Ops recipe

pretrain → post-train
base vs task FT

OpenPI path

pi0 / pi05_base
bring-up in days

Proof points

multi-minute
household skills

Buy / build decision

Prefer start from OpenPI checkpoints + fine-tune

Do NOT reimplement Transfusion+FM from scratch unless you have a research reason

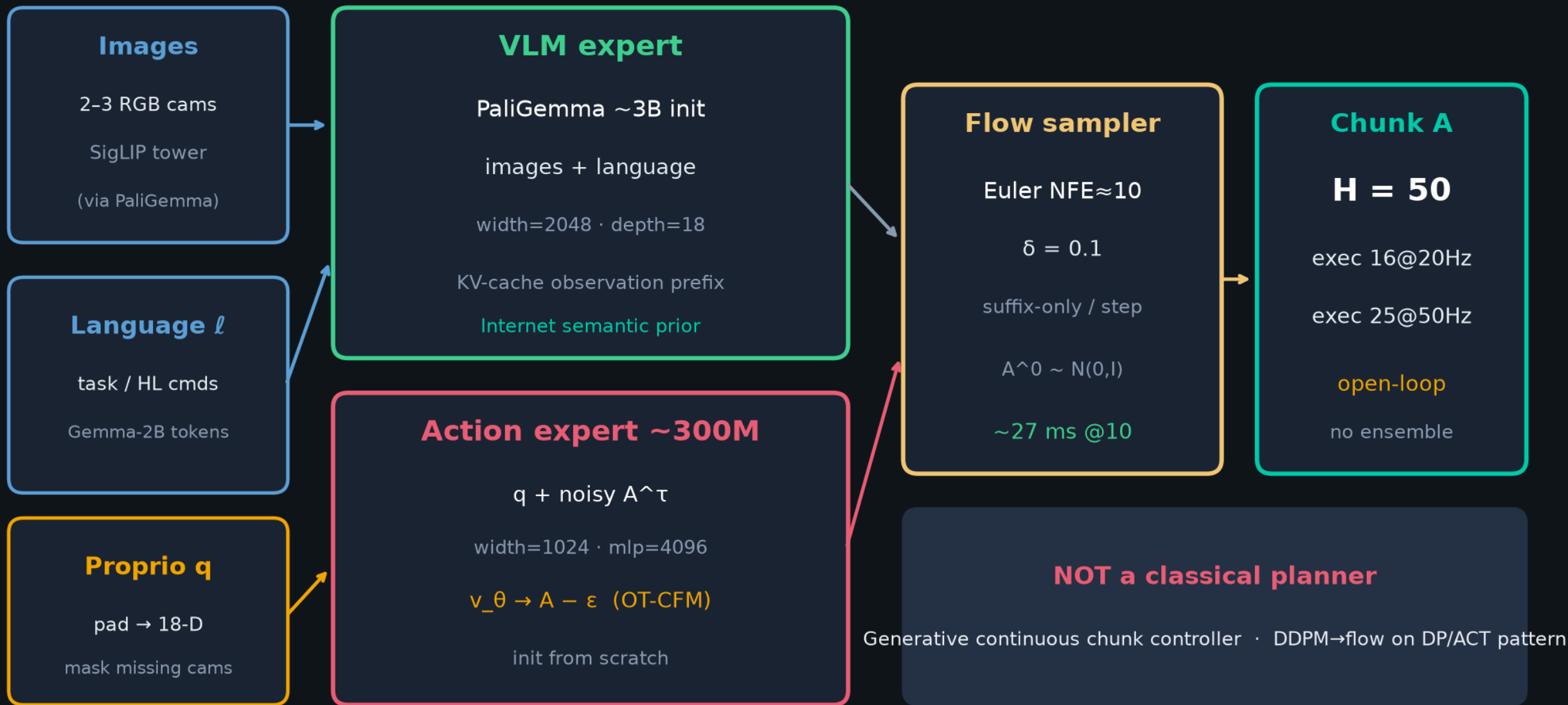
Budget: multi-cam RGB + proprio · GPU adjacent (73–86 ms class) · curated FT hours $\propto$ hardness

Stack snapshot

Backbone	PaliGemma 3B + $\sim 300\text{M}$ FM action expert ($\sim 3.3\text{B}$)
Objective	OT-CFM · $\tau \sim \text{shifted Beta}(1.5, 1)$ · ~ 10 Euler steps
Chunk	$H=50$ train · exec 16@20Hz / 25@50Hz · no ensemble
Latency	~ 73 ms on-board RTX4090 (Table I)
Data	$\sim 10\text{k h}$ proprietary teleop · mixture weight $n^{0.43}$
Embodiments	7 robot configs / 68 tasks + OXE/Bridge/DROID

Architecture · PaliGemma + OT-CFM action expert

~3.3B total · MoE dual-stream (Transfusion-inspired) · blockwise causal attention



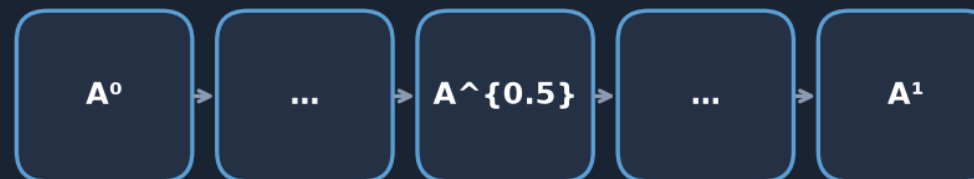
OT-CFM action chunks · DDPM → flow

Linear path $A^\tau = \tau A + (1-\tau)\varepsilon$ · target $u = A - \varepsilon$ · $\tau \sim \text{shifted Beta}(1.5,1)$

Straight OT path in action space



Inference: ~10 Euler steps



$$A^{\{\tau+\delta\}} = A^\tau + \delta \cdot v_\theta(A^\tau, o) \quad \cdot \quad \delta=0.1$$

Chunk → commit → replan (DP/ACT pattern; FM head)

Train H=50

propose full chunk

Execute K

16@20Hz / 25@50Hz

Open-loop

no temporal ensemble

Replan

new infer; RTC later

Orthogonal control knobs

First-class for 2026 · not just the VLM backbone

H

train horizon

paper default 50
sweep {8,16,25,50}

T_a / K

open-loop commit

16@20Hz · 25@50Hz
mid $\approx$ 0.5-1 s sweet

NFE

Euler steps

paper $\sim$ 10
saturates $\sim$ 5-10

async

overlap / RTC

soft-inpaint
exec26 / keep4

Σ

noise corr.

isotropic $\rightarrow$
 $0.5\Sigma + 0.5I$

Latency budget (Table I · RTX 4090 · 3 cams)

Image enc

14 ms

Obs forward

32 ms

$\times 10$ FM

27 ms

On-board

73 ms

+Wi-Fi

$\sim$ 86 ms

Risks & costs

10k h unreproducible

Private teleop corpus dominates pretrain.
Public OXE/OpenPI/sim only for us.
Full pretrain = out of scope.

Open-loop delay seam

No temporal ensembling.
Jerks under obs/network delay.
Mitigate later with RTC soft-inpaint.

Not locomotion-universal

Authors: distant domains
(driving, legged) unproven.
Manipulator-class stack.

Ops / fleet

Multi-cam calibration · GPU adjacency
license & robot ops · FT data cost
scales with task hardness.

Success metrics for a deploy

OOB prompt success

in-mixture skills

1-10 h FT lift

vs ACT / DP / scratch

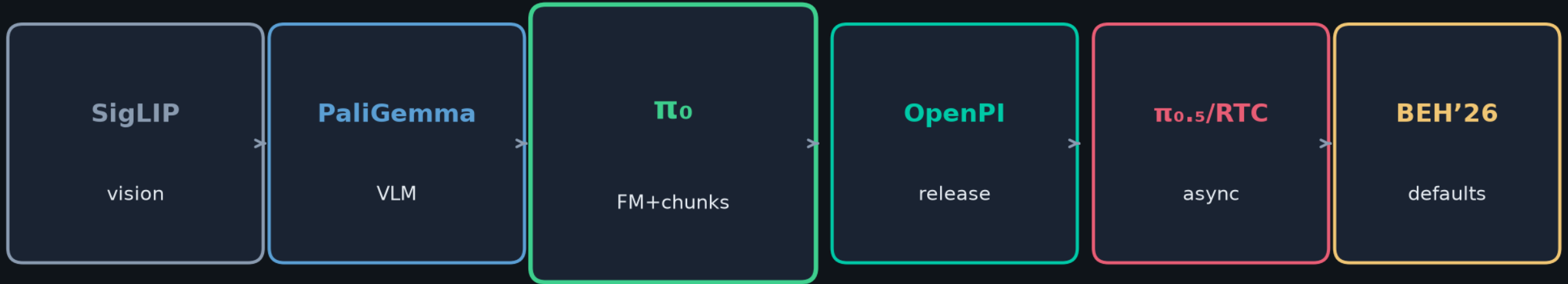
Stable 20-50 Hz

tracking under load

Long-horizon >0.5

progress without teleop

Club spine · one sentence



π_0 popularized flow matching + action chunking on a VLM backbone

as the practical alternative to autoregressive discrete VLAs for dexterous 50 Hz control.

NOT a classical planner · generative continuous chunk controller · knobs: $H \cdot T_a \cdot NFE \cdot \text{async} \cdot \Sigma$

Kevin Black, Noah Brown, Danny Driess, Adnan Esman, Michael Equi, Chelsea Finn, Niccolò Fusar,
Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Sergey Levine,
Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Lucy Xiaoyang Shi, James Tanner, Quan Vuong,
Anna Walling, Haohuan Wang, Ury Zhilinsky
<https://physicalintelligence.company/blog/pi0>

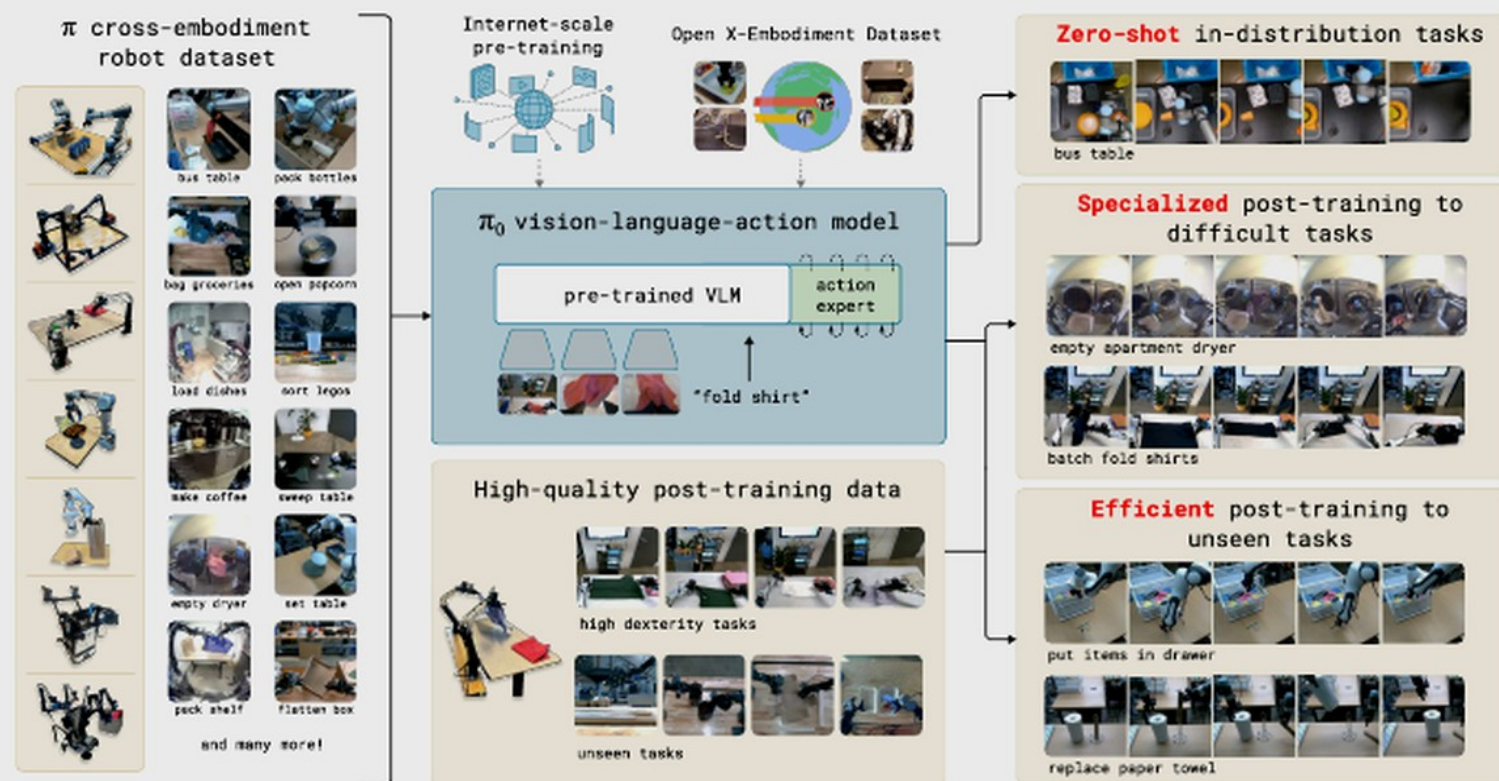


Fig. 1: Our generalist robot policy uses a pre-trained vision-language model (VLM) backbone, as well as a diverse cross-embodiment dataset with a variety of dexterous manipulation tasks. The model is adapted to robot control by adding a separate *action expert* that produces continuous actions via flow matching, enabling precise and fluent manipulation skills. The model can then be used directly to perform tasks based on a prompt, or fine-tuned on high-quality data to enable complex multi-stage tasks, such as folding multiple articles of laundry or assembling a box.

refer to the second set of weights as the *action expert*.

Formally, we want to model the data distribution $p(\mathbf{A}_t|\mathbf{o}_t)$, where $\mathbf{A}_t = [\mathbf{a}_t, \mathbf{a}_{t+1}, \dots, \mathbf{a}_{t+H-1}]$ corresponds to an *action chunk* of future actions (we use $H = 50$ for our tasks), and $\mathbf{o}_t$ is an observation. The observation consists of multiple RGB images, a language command, and the robot’s proprioceptive state, such that $\mathbf{o}_t = [\mathbf{I}_t^1, \dots, \mathbf{I}_t^n, \ell_t, \mathbf{q}_t]$, where $\mathbf{I}_t^i$ is i^{th} image (with 2 or 3 images per robot), ℓ_t is a sequence of language tokens, and $\mathbf{q}_t$ is a vector of joint angles. The images $\mathbf{I}_t^i$ and state $\mathbf{q}_t$ are encoded via corresponding encoders and then projected via a linear projection layer into the same embedding space as the language tokens.

For each action $\mathbf{a}_{t'}$ in the action chunk $\mathbf{A}_t$, we have a corresponding *action token* that we feed through the action expert. During training, we supervise these action tokens using a conditional flow matching loss [28, 32],

$$L^\tau(\theta) = \mathbb{E}_{p(\mathbf{A}_t|\mathbf{o}_t), q(\mathbf{A}_t^\tau|\mathbf{A}_t)} \|\mathbf{v}_\theta(\mathbf{A}_t^\tau, \mathbf{o}_t) - \mathbf{u}(\mathbf{A}_t^\tau|\mathbf{A}_t)\|^2,$$

where subscripts denote robot timesteps and superscripts denote flow matching timesteps, with $\tau \in [0, 1]$. Recent work in high-resolution image [14] and video [38] synthesis has shown that flow matching can achieve strong empirical performance when combined with a simple linear-Gaussian (or optimal transport) probability path [28], given by $q(\mathbf{A}_t^\tau|\mathbf{A}_t) = \mathcal{N}(\tau\mathbf{A}_t, (1-\tau)\mathbf{I})$. In practice, the network is trained by sampling random noise $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, computing the “noisy actions” $\mathbf{A}_t^\tau = \tau\mathbf{A}_t + (1-\tau)\epsilon$, and then training the network outputs $\mathbf{v}_\theta(\mathbf{A}_t^\tau, \mathbf{o}_t)$ to match the denoising vector field $\mathbf{u}(\mathbf{A}_t^\tau|\mathbf{A}_t) = \mathbf{A}_t - \epsilon$. The action expert uses a full bidirectional attention mask, so that all action tokens attend to each other. During training, we sample the flow matching timestep τ from a beta distribution that emphasizes lower (noisier) timesteps. See Appendix B for more details.

At inference time, we generate actions by integrating the learned vector field from $\tau = 0$ to $\tau = 1$, starting with random noise $\mathbf{A}_t^0 \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$. We use the forward Euler integration rule:

$$\mathbf{A}_t^{\tau+\delta} = \mathbf{A}_t^\tau + \delta \mathbf{v}_\theta(\mathbf{A}_t^\tau, \mathbf{o}_t),$$

where δ is the integration step size. We use 10 integration steps (corresponding to $\delta = 0.1$) in our experiments. Note

model, we also trained a similar baseline model that did not use a VLM initialization for ablation experiments. This model, which we refer to as π_0 -small, has 470M parameters, does not use VLM initialization, and has a number of small differences that we found to be helpful for training on our data without VLM initialization, which are summarized in Appendix C. This model is used in our comparisons to evaluate the benefits of incorporating VLM pertaining.

V. DATA COLLECTION AND TRAINING RECIPE

Broadly capable robot foundation models require not only an expressive and powerful architecture, but also the right dataset and, more importantly, the right training *recipe*. In the same way that LLM training is typically divided into pre-training and post-training phases, we employ a multi-stage training procedure for our model. The goal of the pre-training phase is to expose the model to a diverse range of tasks so that it can acquire broadly applicable and general physical capabilities, while the goal of the post-training phase is to provide the model with the ability to skillfully and fluently execute the desired downstream task. Because of this, the requirements for the pre-training and post-training datasets are distinct: the pre-training dataset should cover as many tasks as possible, and within each of those tasks should cover a diversity of behaviors. The post-training dataset should instead cover behaviors that are conducive to effective task execution, which should exhibit a consistent and fluent strategy. Intuitively, the diverse (but lower quality) pre-training data allows the model to recover from mistakes and handle highly varied situations, which might not otherwise occur in the high-quality post-training data, while the post-training data teaches the model to perform the task well.

A. Pre-training and post-training

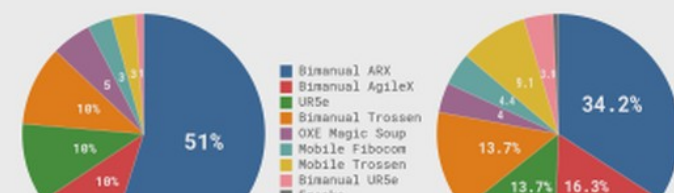




Fig. 12: We evaluate a range of complex and temporally extended tasks. This includes: folding laundry from a bin with a stationary (a) or mobile (b) robot, bussing a real lunch table (c), assembling a box (d), packing eggs into a carton (e), and packing food into a to-go box (f). These tasks require combining dozens of individual behaviors, such as grasping, stacking, folding, and flattening, generalization to a huge variety of object configurations, and complex physical properties, such as deformable objects or flexible cardboard.

are presented in Figure 13. The scoring rubrics are in Appendix E. A score of 1.0 represents a perfect execution, while partial scores correspond to partially completed tasks (e.g., 0.5 indicates that half the objects were bussed correctly). These tasks are very difficult, and we were not able to solve them with other methods. We therefore use these tasks to compare to ablations of our approach, evaluating π_0 after pre-training and fine-tuning, out of the box after pre-training only (“out-of-box”), and training on the fine-tuning data without any pre-training (“scratch”). The results show that π_0 can solve many of these tasks, with our full pre-training and fine-tuning recipe performing best across the board. Note that many of these more difficult tasks show a very large improvement from using the pre-trained model, indicating that pre-training is especially useful with harder tasks. The absolute performance of π_0 varies across the tasks, likely due to differences in task difficulty and the degree to which the tasks are represented in pre-training. We recommend that readers watch the task videos on the [accompanying website](#) for a more complete impression of these tasks and their complexity. We believe that this level of autonomous performance on such challenging

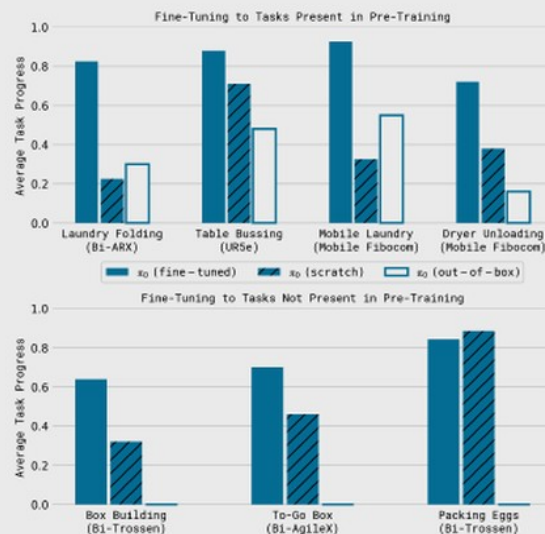


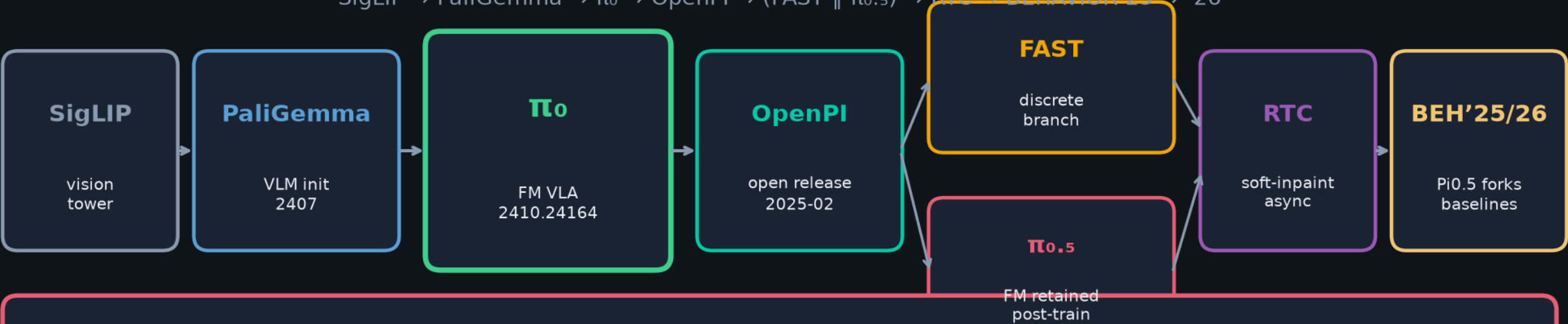
Fig. 13: Post-training results on complex tasks in terms of average scores over 10 trials. The full pre-trained π_0 model attains more than 50% of the maximum score across all of the tasks, and typically outperforms the ablations, with especially significant improvements on the hardest tasks.

Our empirical evaluation studies tasks that combine dexterity, generalization, and temporally extended multi-stage behaviors. Our model incorporates Internet-scale vision-language model (VLM) pre-training with flow matching for representing complex high-frequency action chunks. Our pre-training mixture consists of 10,000 hours of dexterous manipulation data from 7 different robot configurations and 68 tasks, in addition to large amounts of previously collected robot manipulation data from OXE [10], DROID [23], and Bridge [52]. To our knowledge, this represents the largest pre-training mixture ever used for a robot manipulation model. Our fine-tuning experiments include over 20 tasks, where we show that our model outperforms a variety of baselines, including prior VLA models [24] and models designed specifically for dexterous manipulation [57, 9]. We also examine how our post-training recipe can enable highly complex tasks, such as folding multiple articles of clothing from arbitrary initial configurations or assembling boxes.

Our framework broadly resembles the training procedures employed for large language models, which typically consist

Genealogy · CLEAR club line

SigLIP → PaliGemma → π_0 → OpenPI → (FAST || $\pi_{0.5}$) → RTC → BEHAVIOR'25 → '26



CRITICAL CORRECTION

$\pi_{0.5}$ retains flow matching at post-train (OpenPI + $\pi_{0.5}$ paper)

Secondary blogs claiming “FAST replaced FM” = FALSE · hybrid FAST pretrain + FM post-train

Pivot this paper locks

FM VLA + VLM init + action expert + 50 Hz chunks at ~10k h cross-embodiment scale

Stakeholder · close

Start from OpenPI. Treat π_0 as the reference VLA template.

Budget multi-cam + 73–86 ms GPU class · risk: 10k h unreproducible

Q&A · Sep 28 · π_0 / 2410.24164

π_0 : A Vision-Language-Action Flow Model for General Robot Control

Physical Intelligence

Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Sergey Levine, Adrian Li Bell, Mohith Mohukuri, Suraj Nair, Karl Pertsch, Lucy Xiaoqiang Shi, James Tapper, Quan Vuong